BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 13

Functions Part 3

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3-4
	1.1 Objectives	-	-	
2	Function With Operators	-	1	5-8
3	Function And Decision Statements	-	2	8-11
4	Function And Loop Operators	-	3	11-12
3	Summary	-	-	13
4	Terminal Questions	-	-	14
5	Answers To Self Assessment Questions	-	-	14
6	Answers To Terminal Questions	-	-	14-15

1. INTRODUCTION

In C programming, operators, functions, and decision statements are fundamental concepts that enable developers to write efficient and structured code. Operators are symbols that perform specific operations on operands, such as arithmetic, bitwise, logical, and relational operations. Functions, on the other hand, encapsulate blocks of code that perform specific tasks and can be called from other parts of the program. Functions can interact with operators to manipulate data effectively within a program, providing modularity and code reusability.

While C does not support native operator overloading like C++, functions can emulate operator-like behavior for custom data types, providing similar functionality. Unary operators act on a single operand, and functions can be used to implement custom behaviors for unary operations. Bitwise operators manipulate individual bits of integer operands, and functions can encapsulate bitwise operations to perform tasks such as setting, clearing, or toggling specific bits. Logical operators perform logical operations on boolean values, and functions can incorporate logical operators to make decisions and control program flow based on boolean conditions.

The conditional (ternary) operator provides a compact way to express conditional statements within a single line of code. Functions can utilize the conditional operator to implement conditional expressions efficiently. Moreover, decision statements such as if, else if, and else, allow functions to make choices based on certain conditions, enhancing the flexibility and versatility of function behavior. Additionally, loop operators like for, while, and do-while enable functions to execute a block of code repeatedly, providing the ability to iterate over data and perform calculations or tasks multiple times. Understanding these concepts is essential for writing efficient and structured C programs, empowering developers to manipulate data and control program flow effectively.

1.1. Objectives:

At studying this unit, you should be able to:

- *Learn how to simulate operator overloading in C by defining functions with operator-like behavior for custom data types.*
- *Gain proficiency in writing functions that incorporate decision statements to perform different actions based on specific conditions.*
- *Gain proficiency in implementing functions that utilize loop operators to iterate over data, perform calculations, or execute tasks multiple times.*

INSPIRED BY LIFE

2. FUNCTION WITH OPERATORS

Operator overloading is not directly supported in standard C language. Unlike languages like C++, C# or Python, where you can define custom behaviors for operators like +, -, *, etc., based on the types of their operands, C does not provide this feature. In C, operators have fixed meanings determined by the language specification, and they cannot be redefined or overloaded. Therefore, you cannot define custom behaviors for operators when working with user-defined types in C.

However, you can simulate operator overloading in C by defining functions that perform the desired operations and then calling those functions explicitly. For example, instead of overloading the + operator for custom types, you can define a function like add() to perform addition operations on objects of those types. In C programming, operator overloading is not directly supported as it is in languages like C++. However, we can still demonstrate unary operators, bitwise operators, logical operators, and the conditional (ternary) operator in a C program. Here's a program showcasing each of these concepts:

```
#include <stdio.h>
// Unary operator example
void unaryOperatorExample() {
   int x = 10;
   printf("Unary Operator Example:\n");
   printf("Original value of x: %d\n", x);
   // Unary operators
   printf("x after unary increment (++x): %d\n", ++x);
   printf("x after unary decrement (--x): %d\n", --x);
}


// Bitwise operator example
void bitwiseOperatorExample() {
```

```
    int a = 10, b = 5;
    printf("\nBitwise Operator Example:\n");
    printf("a: %d, b: %d\n", a, b);
    // Bitwise AND
    printf("a & b: %d\n", a & b);
    // Bitwise OR
    printf("a | b: %d\n", a | b);
    // Bitwise XOR
    printf("a ^ b: %d\n", a ^ b);
    // Bitwise NOT
    printf("~a: %d\n", ~a);
}
// Logical operator example
void logicalOperatorExample() {
    int x = 10, y = 5;
    printf("\nLogical Operator Example:\n");
    printf("x: %d, y: %d\n", x, y);
    // Logical AND
    printf("x && y: %d\n", x && y);
    // Logical OR
    printf("x || y: %d\n", x || y);
    // Logical NOT
    printf("!x: %d, !y: %d\n", !x, !y);
}
```

```
// Conditional (Ternary) operator example
void conditionalOperatorExample() {
    int a = 10, b = 5;
    int max = (a > b) ? a : b;
    printf("\nConditional (Ternary) Operator Example:\n");
    printf("a: %d, b: %d\n", a, b);
    printf("Maximum value: %d\n", max);
}
int main() {
    unaryOperatorExample();
    bitwiseOperatorExample();
    logicalOperatorExample();
    conditionalOperatorExample();
    return 0;
}
```

This C program showcases different types of operators:

Unary Operator Example:

Demonstrates pre-increment and pre-decrement operations (++x, --x) on the variable 'x'.

Bitwise Operator Example:

Shows bitwise AND (&), OR (|), XOR (^), and NOT (~) operations between two integers 'a' and 'b'.

Logical Operator Example:

Displays logical AND (&&), OR (||), and NOT (!) operations between two integers 'x' and 'y'.

Conditional (Ternary) Operator Example:

Utilizes the ternary operator (?:) to find the maximum value between 'a' and 'b' and assigns it to 'max'. Finally, the main() function calls each of these examples, printing the results to the consol

SELF-ASSESSMENT QUESTIONS - 1

1. In C programming, which operator is used to perform logical AND between two expressions?
   (a) &&
   (b) ||
   (c) &
   (d) |
2. Which of the following operators is used to perform logical negation in C?
   (a) +
   (b) -
   (c) !
   (d) ~

3. FUNCTION AND DECISION STATEMENTS

In C programming, decision statements (such as if, else if, and else) can be used within function bodies to make logical decisions based on input parameters or other variables. Functions can perform different actions or return different results depending on the outcome of decision statements. They provide a way to customize function behavior based on specific conditions, enhancing flexibility and versatility in program execution. Here's a simple example illustrating the usage of decision statements within a function:

Example:

```
#include <stdio.h>
```

```
// Function to determine the maximum of two numbers
int max(int a, int b) {
   if (a > b) {
      return a;
   } else {
      return b;
   }
}
int main() {
   int num1 = 10, num2 = 20;
   // Call the max function and print the result
   printf("Maximum number: %d\n", max(num1, num2));
   return 0;
}
```

In this example, the max() function takes two integer parameters a and b and uses an if statement to compare them. If a is greater than b, the function returns a, otherwise, it returns b. The main() function calls max() with two numbers and prints the maximum value. This illustrates how decision statements can be effectively used within functions to perform different actions based on specific conditions.

Example:

```
#include <stdio.h>
// Function to determine the maximum of three numbers
int maxOfThree(int a, int b, int c) {
   if (a >= b && a >= c) {
      return a;
```

```
    } else if (b >= a && b >= c) {
        return b;
    } else {
        return c;
    }
}

int main() {
    int num1 = 10, num2 = 20, num3 = 15;
    // Call the maxOfThree function and print the result
    printf("Maximum number: %d\n", maxOfThree(num1, num2, num3));
    return 0;
}
```

In this extended example, the maxOfThree() function takes three integer parameters a, b, and c and uses decision statements to determine the maximum among them. It compares each number with the others and returns the maximum value. The main() function calls maxOfThree() with three different numbers (num1, num2, num3) and prints the maximum value. This demonstrates how decision statements can be used within functions to perform more complex tasks based on specific conditions.

SELF-ASSESSMENT QUESTIONS - 2

3. Functions with decision statements allow for:
   (a) Repeating a block of code
   (b) Customizing function behavior based on specific conditions
   (c) Declaring arrays and structures
   (d) Printing output to the console
4. Which keyword is used to introduce a decision statement in C programming?
   (a) decision
   (b) if
   (c) choose
   (d) pick

4. FUNCTION AND LOOP OPERATORS

Function and Loop Operators" in C programming involves using loop constructs within functions to perform repetitive tasks efficiently. Functions encapsulate blocks of code that perform specific tasks, and loop operators allow executing a block of code repeatedly based on certain conditions. Here's an example demonstrating the usage of loop operators within a function in C:

```
#include <stdio.h>
// Function to calculate the factorial of a number using a loop
int factorial(int n) {
  int result = 1;
  for (int i = 1; i <= n; i++) {
    result *= i;
  }
```

```
    return result;
}
int main() {
    int num = 5;
    // Call the factorial function and print the result
    printf("Factorial of %d is: %d\n", num, factorial(num));
    return 0;
}
```

In this example, the factorial() function takes an integer parameter n and uses a for loop to calculate the factorial of n. The loop iterates from 1 to n, multiplying each integer along the way to compute the factorial. The function then returns the final result.

The main() function calls factorial() with a value of num (in this case, 5) and prints the factorial result. This demonstrates how loop operators, in this case, the for loop, can be effectively used within functions to perform repetitive tasks, such as calculating factorials, in C programming.

SELF-ASSESSMENT QUESTIONS - 3

5. What is the purpose of using loop operators within functions in C programming?
   (a) To define variables
   (b) To perform mathematical operations
   (c) To execute a block of code repeatedly based on certain conditions
   (d) To print output to the console
6. Which loop operator allows executing a block of code at least once, regardless of the loop condition?
   (a) for loop
   (b) while loop
   (c) do-while loop
   (d) break statement

4. SUMMARY

In C programming, operators, functions, and decision statements are foundational concepts that enable efficient and structured code development. Operators are symbols performing specific operations on operands, including arithmetic, bitwise, logical, and relational operations. Functions encapsulate code blocks for specific tasks, enhancing modularity and code reusability. While C lacks native operator overloading like C++, functions can mimic operator-like behavior for custom data types.

Unary operators act on a single operand, and functions can implement custom behaviors for unary operations. Bitwise operators manipulate individual bits of integer operands, with functions encapsulating bitwise operations for tasks like setting or clearing specific bits. Logical operators perform boolean operations, aiding decision-making within functions. The conditional operator offers concise conditional expressions.

Decision statements (if, else if, else) allow functions to make choices based on conditions, enhancing flexibility. Loop operators (for, while, do-while) enable repetitive code execution within functions, facilitating data iteration and task repetition. Understanding these concepts is vital for efficient and structured C programming, empowering developers to manipulate data and control program flow effectively.

INSPIRED BY LIFE

5. TERMINAL QUESTIONS

1. Discuss how loop operators enable functions to execute a block of code repeatedly and provide examples demonstrating their practical application..
2. Discuss the concept of functions with arrays in C programming. Explain how functions can accept arrays as arguments and return arrays as results.
3. Provide examples to illustrate how decision statements can be used to make choices based on different conditions within functions.

6. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. (a) &&
2. (c) !
3. (b) Customizing function behavior based on specific conditions
4. (b) if
5. (c) To execute a block of code repeatedly based on certain conditions
6. (c) do-while loop

7. ANSWERS TO TERMINAL QUESTIONS

1. (Refer Section 4 for more details)
2. (Refer Section 2 for more details)
3.
```
#include <stdio.h>
// Function to check if a number is even or odd
void checkEvenOrOdd(int num) {
    if (num % 2 == 0) {
        printf("%d is even.\n", num);
    } else {
        printf("%d is odd.\n", num);
    }
}
```

```
int main() {
    checkEvenOrOdd(5);
    checkEvenOrOdd(10);
    return 0;
}
```